

kongtree1.sql 数据导入报告

- 日期：2026-08-06
- 生产环境：192.168.1.112，数据库 kzjp01
- 数据来源：旧系统导出文件 kongtree1.sql（约 870MB，Navicat 导出，源库 kongtree）
- 操作原则：仅对比生产库；文件中有、生产没有的数据补入；生产独有数据保留不动；不涉及本地 MySQL

1. 背景与目标

旧系统导出的 kongtree1.sql
含完整家谱底库及后期补遗录入数据。当前编修平台生产库此前基于较早版本导入，人员规模少于该文件。

本次目标：

1. 对比 dump 与生产库主键差异；
2. 将 dump 中缺失的数据导入生产；
3. 处理导入后的性能与父子关系数据质量问题。

2. 对比方法

1. 从生产导出各表主键集合；
2. 流式扫描 kongtree1.sql 中的 INSERT 语句；
3. 按主键求差集，生成兼容生产表结构的补数 SQL；
4. 上传至生产执行。

表结构差异（导入时已裁剪）：

表	dump 多出的列	处理
tb_people	F_NATION	导入时去掉
tb_people_info	F_SIGN	导入时去掉
tb_people_relation	F_CHILDREN	导入时去掉
tb_module	F_ASSETS	导入时去掉

孤立关系过滤：dump 中有 66 条 tb_people_relation 无对应人员记录，未导入。

相关本地产物（工作目录）：

- tmp-sync/diff-summary.txt 一 对比汇总
- tmp-sync/missing-from-kongtree1.sql 一 补数 SQL（约 14MB）
- tmp-sync/diff_and_build_import.py 一 对比/生成脚本

3. 对比结果与导入数量

3.1 核心家谱表

表	生产原有	dump	本次补入	生产独有（保留）
tb_people	1,708,404	1,735,274	26,953	83
tb_people_info	1,708,404	1,735,274	26,953	83
tb_people_relation	1,708,404	1,735,423	26,953（另过滤 66 条孤立边）	

导入后人员三表均为：

1,735,357 = dump 人员数 1,735,274 + 生产独有 83

抽样验证（均已存在）：

- 1708360 昭貴
- 1735274 祥雨
- 1735423 維俊丞

3.2 附属/业务表

表	生产原有	dump	本次补入
tb_people_check	4,055	5,125	1,070
tb_people_temp	3,550	4,247	698
tb_log	3,747	4,107	360
tb_module	67	73	6
tb_pdf_file	64	68	4
tb_download_record	116	122	6
tb_user	5	7	3（kongpeng / liyuan / xiaoyuan）
tb_user_role	10	8	3
tb_role_module	111	251	148
tb_branch	450	450	0（已对齐）
tb_people_continued	47,586	47,586	0（已对齐）
tb_role	5	4	0

导入后已校正相关表 AUTO_INCREMENT（此前曾卡在 1708411，现与 MAX(F_ID)+1 一致）。

4. 导入后性能问题

4.1 现象

补数完成后页面加载变慢。

4.2 根因

指标	导入后（调优前）	说明
核心表体积（people/info/relation/ext）	约 1136 MB	数据+索引
innodb_buffer_pool_size	128 MB	默认过小
Innodb_buffer_pool_wait_free	63,752	缓冲池打满后大量等待
主机内存	15 GiB	有充足余量

批量写入后热页被挤出内存，磁盘读增多，表现为查询/加载变慢。仓库 Docker 配置中本就有 innodb_buffer_pool_size = 2G，但生产原生 MySQL 未生效。

4.3 处理

在生产执行 ~/kong-jiapu/deploy/apply-mysql-perf.sh：

- 写入 /etc/mysql/mysql.conf.d/zz-kongjp-perf.cnf
- 将 innodb_buffer_pool_size 调整为 2G
- 重启 MySQL 并 ANALYZE TABLE 核心表

调优后确认：

- innodb_buffer_pool_size = 2147483648（2G）
- 空闲缓冲页约 10 万+
- 列表/按父取子类查询恢复到亚毫秒~毫秒级

仓库内对应文件：

- deploy/mysql/zz-kongjp-perf.cnf
- deploy/apply-mysql-perf.sh

5. 父子关系断链问题（F_PARENT_ID = 0）

5.1 现象

列表单查「祥满」可见父亲「繁营」；姓名 + 父亲姓名联合查询却无结果。

5.2 根因

示例人员 1733472（祥满，嶺南派/广东广州）：

字段	值
F_PARENT_NAME	繁營（列表展示用此字段）
F_PARENT_ID	0（未挂真实父亲节点）

联合查询原逻辑只做：

relation.F_PARENT_ID → tb_people.F_NAME

F_PARENT_ID = 0 时 INNER JOIN 失败，故无结果。

该情况来自旧系统原文，非本次导入丢失。dump 中即为：

```
INSERT INTO `tb_people_relation` VALUES ('1733472', '', '0', '繁營', '', '0', '', 'fan ying', null);
```

对应人员为旧系统补遗/录入（如 孔曉媛，F_ORIGINAL_DATA='0'），常只填父名不选父亲节点。

分布：

范围	有父名且 F_PARENT_ID=0
导入前已有数据	0
本次导入数据	3,807

5.3 数据修复（已在生产执行）

仅在唯一匹配时补链，规则分两轮：

1. 父名相同 + 世代 = 子代世代 - 1 + 派户支完全相同 → 1,380 条
2. 再加上地址完全相同 → 再补 747 条

项目	数量
成功补链	2,127
仍无法唯一确定（暂不自动挂）	1,680

示例「祥满 / 繁營」已修复：F_PARENT_ID = 1733471。

回滚备份表：kzjp01.tmp_parent_id_repair_20260806

（字段含 people_id / old_parent_id / new_parent_id / parent_name / matched_by / repaired_at）

5.4 查询兼容（代码已改，待部署）

web/src/lib/people.ts 中父亲/祖父条件改为同时匹配：

- tb_people.F_NAME（经 F_PARENT_ID）
- tb_people_relation.F_PARENT_NAME（旧谱姓名无 ID 的情况）

部署后，剩余 1,680 条断链也可按父名联查命中。

6. 对后续录入的影响

问题	是否影响本平台新录入	说明
历史 F_PARENT_ID=0	否	新录入走选定父亲 parentId，会同时写 ID 与姓名
已自动补链的 2,127 条	极低风险	仅唯一匹配；误挂可按备份表回滚

剩余 1,680 条断链	不阻塞录入	展开子代/按 ID 挂靠可能看不到，待查或人工补链
查询兼容父名	不影响写入	只改检索，不改保存逻辑

本平台世系新增、批量导入等在父亲重名时要求选定具体父亲，不会系统性再现「只填父名、ID 为 0」。

7. 当前生产状态（截至报告撰写）

项	值
tb_people 行数	1,735,357
InnoDB 缓冲池	2G
父名有、父 ID 为 0（剩余）	1,680
父 ID 补链备份表	tmp_parent_id_repair_20260806（2,127 行）
查询兼容父名代码	已改本地，尚未部署生产

8. 待办建议

1. 提交并部署 people.ts 父名联查兼容，使剩余断链可被联合查询命中。
2. （可选）对剩余 1,680 条提供人工核对清单，或按更严规则（谱号、住址模糊等）分批补链。
3. （可选）录入校验：若提交含父名却无 parentId，直接拒绝，防止新脏数据。
4. 保留 tmp_parent_id_repair_20260806 一段时间后再决定是否归档删除。

9. 结论

本次从 kongtree1.sql 向生产补入约 2.7 万人员及配套 info/relation，以及审核、日志、模块等附属数据；生产独有 83 人未覆盖。

导入后因 MySQL 缓冲池过小导致变慢，已调至 2G 并恢复正常。导入数据中 3,807 条「有父名无父 ID」为旧系统补遗特征；已安全自动补链 2,127 条，余 1,680 条需查询兼容或人工处理。本平台正常录入路径不会系统性再产生同类断链。